Build2D

2D Floor Planning Desktop Application

Java Desktop Application with Firebase Authentication

Project Documentation

March 17, 2026

Developed in Java with Swing UI Framework

Contents

1 Project Overview **3**
- 1.1 The Problem This Project Solves . . . 3
- 1.2 Why This Project Is Real-World Relevant . . . 3
- 1.3 Who Uses This and When . . . 3

2 High-Level Architecture **4**
- 2.1 Main Components . . . 4
- 2.2 How Files Interact . . . 4
- 2.3 Simple Mental Model . . . 4

3 Language & Tools Explained **5**
- 3.1 What is Java? . . . 5
- 3.2 Why Java for This Project? . . . 5
- 3.3 Key Tools in This Project . . . 5

4 File-by-File Explanation **5**
- 4.1 Login.java - The Entry Gate . . . 5
- 4.2 GoogleAuthConfig.java - The Security Manager . . . 6
- 4.3 RoomPlan2.java - The Main Canvas (The Heart of the App) . . . 6
  - 4.3.1 A. The Drawing Canvas . . . 6
  - 4.3.2 B. The Room Class (Nested inside RoomPlan2) . . . 7
  - 4.3.3 C. The Furniture Class (Nested inside RoomPlan2) . . . 7
  - 4.3.4 D. The Door Class (Nested inside RoomPlan2) . . . 7
  - 4.3.5 E. The Save/Load System . . . 7
  - 4.3.6 F. Mouse Interaction . . . 8
  - 4.3.7 G. Main Method . . . 8
- 4.4 test.java - An Alternative/Test Version . . . 8
- 4.5 pom.xml - The Recipe for Building the App . . . 9
- 4.6 Config Files (Mentioned but Not Shown) . . . 9
- 4.7 images/ Folder . . . 10
- 4.8 README.md & PRIVACY_POLICY.md . . . 10

5 Core Logic Walkthrough **10**
- 5.1 Scenario: User Creates a Floor Plan . . . 10
- 5.2 Step-by-Step Process . . . 11

6 Interview-Ready Explanation **11**
- 6.1 Common Interview Questions & Strong Answers . . . 12
  - 6.1.1 Q1: "Why did you choose Java over Python or JavaScript?" . . . 12
  - 6.1.2 Q2: "How does the collision detection work?" . . . 12
  - 6.1.3 Q3: "Why did you create a custom .2ds file format instead of JSON?" 12
  - 6.1.4 Q4: "What authentication method do you recommend—Firebase or Google OAuth?" . . . 12
  - 6.1.5 Q5: "How would you scale this to handle 1000 users saving plans simultaneously?" . . . 13
  - 6.1.6 Q6: "What would you change if you rewrote this today?" . . . 13

6.2 Key Technical Talking Points 14

7 Simplification for Interviews **14**

8 Final Checklist **15**

9 Future Enhancements **15**

9.1 Short-term Improvements 15

9.2 Long-term Vision 15

10 Conclusion **16**

A Appendix: Complete File Structure **17**

1 Project Overview

Information

Project Name: Build2D
Type: Desktop Application
Language: Java
UI Framework: Swing with FlatLaf
Authentication: Firebase & Google OAuth
File Format: Custom .2ds

1.1 The Problem This Project Solves

Build2D is a desktop application that solves the problem of creating and planning 2D floor layouts visually. Instead of drawing floor plans on paper or struggling with complex CAD software, this application enables users to:

- **Quickly design rooms** in a building with intuitive drag-and-drop interface
- **Position and arrange** them on a grid-based canvas
- **Add furniture and fixtures** including doors, windows, and decorative items
- **Save designs** for later editing and iteration

1.2 Why This Project Is Real-World Relevant

Floor planning is essential for multiple professional and personal use cases:

User Type	Use Case
Architects & Interior Designers	Creating initial designs before detailed CAD work
Property Managers	Planning renovations or space reorganization
Real Estate Agents	Showing property layouts to potential clients
Home Owners	Visualizing renovation ideas before working with professionals

Table 1: Target Users and Their Use Cases

Key Advantage: The tool makes floor planning accessible to non-professionals—you don't need expensive software like AutoCAD; you just need this simple Java application.

1.3 Who Uses This and When

- **Primary Users:** Anyone planning a building layout (homeowner, designer, architect)
- **Use Case:** When you need to quickly prototype a floor plan and iterate on room placement

- **Scenario Example:** *"I'm renovating my apartment and want to see if I can fit a home office in the corner of my bedroom"*

2 High-Level Architecture

2.1 Main Components

Think of the application like a layered sandwich:

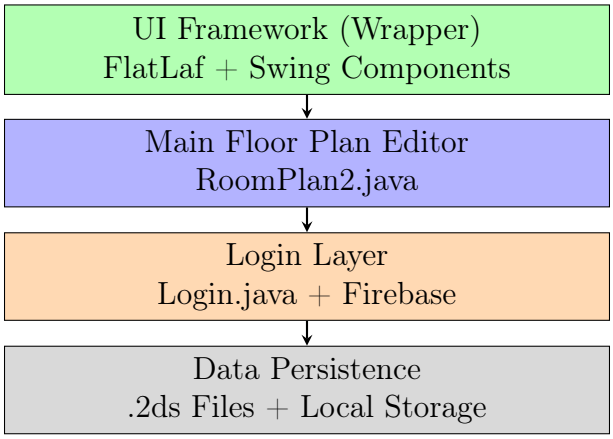

Figure 1: Build2D Application Architecture Layers

2.2 How Files Interact

1. User starts application
2. `Login.java` checks credentials (with Firebase & Google Auth)
3. `RoomPlan2.java` opens - main canvas for drawing
4. User creates rooms (`Room` class), adds furniture (`Furniture` class)
5. User clicks "Download" → saves to `.2ds` file
6. Next time, click "Open File" → `RoomPlan2` loads your design

2.3 Simple Mental Model

Information

Imagine the application as a **whiteboard with grid lines**:

- The grid helps you place rooms accurately
- Each room is a colored rectangle with properties (width, height, type)
- You can draw on the whiteboard, but when you close it, your drawing disappears—unless you save it to a `.2ds` file
- The login screen ensures only authorized people access the planner

3 Language & Tools Explained

3.1 What is Java?

Java is a "write once, run anywhere" programming language created in 1995.

Characteristic	Description
Cross-platform	Same code runs on Windows, Mac, Linux
Verbose but clear	You write more code, but it's very readable
Object-oriented	Organizes code around "objects" (like Room, Furniture)
Compiled then Interpreted	Converts to intermediate code that runs on JVM

Table 2: Key Characteristics of Java

3.2 Why Java for This Project?

- **Desktop UI:** Java has Swing/AWT—built-in libraries for creating windows, buttons, and graphics
- **Cloud Integration:** Integrated with Firebase & Google Auth for secure login
- **Cross-platform:** Users on any OS can run the app without modification
- **Professional Standard:** Used by companies for mission-critical applications

3.3 Key Tools in This Project

Tool	Purpose
Maven (pom.xml)	Package manager - downloads libraries automatically
FlatLaf	Makes the UI look modern (replaces clunky Java defaults)
Firebase Admin SDK	Handles user registration and authentication
Google OAuth	Lets users log in with their Google account
Swing Components	JPanel, JButton, JFrame - basic UI blocks

Table 3: Development Tools and Libraries

4 File-by-File Explanation

4.1 Login.java - The Entry Gate

Purpose: This is the first screen users see.

What it does:

- Shows a login form with username/password fields
- Validates credentials against Firebase (a cloud database service)
- Offers two login methods:
  - **Manual:** Enter admin/admin credentials
  - **Google OAuth:** Click "Login with Google"

Code Example

```
// User types username/password
// Clicks Login
// Code sends data to Firebase to verify
// If valid     opens RoomPlan2 (main editor)
// If invalid     shows error popup
```

Listing 1: Login Flow Simplified

Connection: This is the gateway. Until you pass this screen, you can't access the floor planner.

4.2 GoogleAuthConfig.java - The Security Manager

Purpose: Manages secure authentication with Google's servers.

What it does:

- Sets up the security credentials to connect to Google
- Creates an OAuth flow (a standardized way to safely log in)
- Verifies that login tokens are legitimate
- Handles the entire Google login process

Key concepts inside:

- **Client ID & Secret:** Like a passport for this app to talk to Google
- **OAuth Flow:** A secure protocol where you never directly share your Google password
- **Token Verification:** Ensures the user really logged in with Google

4.3 RoomPlan2.java - The Main Canvas (The Heart of the App)

Purpose: This is the core editing interface where all the magic happens.

4.3.1 A. The Drawing Canvas

- A grid-based whiteboard where rooms appear as colored rectangles
- Users can drag rooms around with the mouse
- Shows a legend on the left (color key: Green=Bedroom, Red=Kitchen, etc.)

4.3.2 B. The Room Class (Nested inside RoomPlan2)

Technical Details

What is a Room? A room is defined by: position (X, Y), dimensions (width, height), color, and type. It can contain doors and windows. It can detect if your mouse is hovering over it.

Code Example

```
Room bed = new Room(100, 50, 300, 200, Color.GREEN, "Bedroom");
// Position: X=100, Y=50
// Dimensions: 300w    200h
// Color: Green
// Type: Bedroom
```

Listing 2: Room Object Creation

4.3.3 C. The Furniture Class (Nested inside RoomPlan2)

What is Furniture? Items placed inside rooms: beds, tables, sofas, etc. Each furniture piece has: position, type name, and a visual image.

4.3.4 D. The Door Class (Nested inside RoomPlan2)

What is a Door? Doors are attached to room walls (North, South, East, West). They have a collision detection system:

- You can't place a door where a window already exists
- Ensures realistic floor plans

4.3.5 E. The Save/Load System

Custom File Format (.2ds files):

Code Example

```
2DS_FLOOR_PLAN
VERSION 1.0
ROOM_COUNT 3
ROOM
  TYPE Bedroom
  POSITION 100 50
  DIMENSIONS 300 200
END_ROOM
ROOM
  TYPE Kitchen
  POSITION 400 50
  DIMENSIONS 250 300
END_ROOM
...
END_FLOOR_PLAN
```

Listing 3: .2ds File Format Example

How it works:

- **Download button:** Saves current floor plan to .2ds file (custom format)
- **Open File button:** Reads .2ds file and reconstructs rooms on canvas
- **Reset button:** Clears everything to start fresh

4.3.6 F. Mouse Interaction

- **Click on room:** Selects it (shows with border)
- **Drag room:** Moves it around the canvas
- **Collision detection:** Prevents rooms from overlapping

4.3.7 G. Main Method

Code Example

```java
public static void main(String[] args) {
    UIManager.setLookAndFeel(new FlatMacLightLaf()); // Modern look
    SwingUtilities.invokeLater(() -> {
        Login login = new Login(); // Show login first
        login.setVisible(true);
    });
}
```

Listing 4: Application Entry Point

4.4 test.java - An Alternative/Test Version

Purpose: This is a simplified version or experimental version of RoomPlan2, likely:

- Used for testing new features
- An older iteration of the main application
- A backup or demo version

Connection: Not used in production; kept for reference.

4.5 pom.xml - The Recipe for Building the App

Purpose: Tells Maven (the build tool) what ingredients (libraries) this app needs.

Code Example

```xml
<dependencies>
    <!-- Modern UI Look -->
    <dependency>
        <groupId>com.formdev</groupId>
        <artifactId>flatlaf</artifactId>
        <version>3.2</version>
    </dependency>

    <!-- Firebase Authentication -->
    <dependency>
        <groupId>com.google.firebase</groupId>
        <artifactId>firebase-admin</artifactId>
        <version>9.2.0</version>
    </dependency>

    <!-- Google OAuth -->
    <dependency>
        <groupId>com.google.auth</groupId>
        <artifactId>google-auth-library-oauth2-http</artifactId>
        <version>1.19.0</version>
    </dependency>
</dependencies>
```

Listing 5: Key Dependencies in pom.xml

How it works:

1. Developer runs: `mvn clean install`
2. Maven reads pom.xml
3. Maven downloads all libraries from the internet
4. Compiles the code with all libraries
5. Creates a runnable application

4.6 Config Files (Mentioned but Not Shown)

The code references:

- `config/client_secrets.json`: Google OAuth credentials (hidden for security)

- `config/key.json`: Firebase service account key (hidden for security)

Important: These are NOT included in the project for security reasons—they'd be added during deployment.

4.7 images/ Folder

Contains furniture PNG images: `bed.png`, `chair.png`, `sofa.png`, etc. When you add furniture to a room, the app draws these images on the canvas so it's visual, not just text.

4.8 README.md & PRIVACY_POLICY.md

- **README:** Installation & usage instructions
- **PRIVACY_POLICY:** Legal document explaining how user data is handled

5 Core Logic Walkthrough

5.1 Scenario: User Creates a Floor Plan

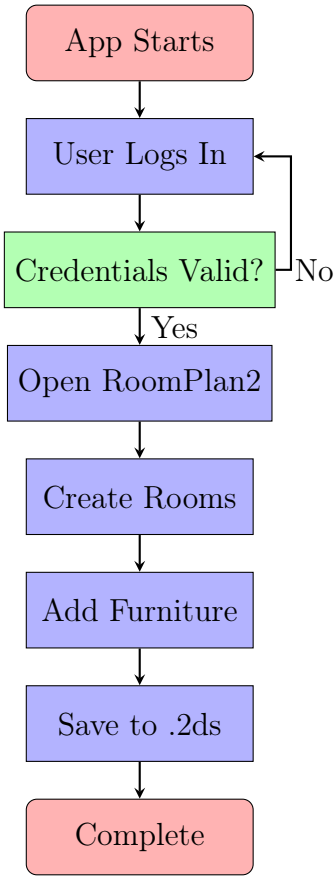

Figure 2: User Flow Diagram

5.2 Step-by-Step Process

1. **App Starts:** User clicks app executable → `main()` runs → Sets FlatLaf theme → Opens Login window
2. **User Logs In:** User enters credentials → Login.java initializes Firebase → Sends credentials → Firebase validates → Opens RoomPlan2
3. **Create First Room:** User clicks "Room Type" → Dropdown appears → User selects type → Color auto-assigned
4. **Set Dimensions:** Dialog box appears → User enters Width and Height → Values stored
5. **Set Position:** Dialog asks for X and Y coordinates → User enters values → Position stored
6. **Add Room to Canvas:** Code creates Room object → Adds to list → Calls `repaint()` → User sees rectangle
7. **Add Furniture:** User chooses furniture type → Clicks inside room → Image loaded → Drawn on canvas
8. **Add Door:** User right-clicks room → Selects wall → Collision check → Door created
9. **Save to File:** User clicks "Download" → File chooser opens → User names file → Code writes to .2ds
10. **Close and Reopen:** User opens app → Clicks "Open File" → Code reads file → Reconstructs rooms → Canvas redraws

6 Interview-Ready Explanation

Interview Ready

Your 90-Second Pitch

"I built Build2D, a desktop floor planning application in Java. It solves the problem of quickly visualizing and prototyping 2D room layouts without expensive software like AutoCAD. Here's how it works:

Users log in securely using Firebase or Google OAuth. Once authenticated, they enter a grid-based canvas. They can create multiple room types—bedrooms, kitchens, bathrooms—each with a distinct color for visual clarity. For each room, they specify dimensions and position. The app prevents overlapping rooms and includes collision detection for doors and windows.

The architecture is straightforward: Login screen → Secure authentication → Main canvas editor. The core logic involves a Room class that stores position, dimensions, color, and type. Inside rooms, users can add Furniture (drawn as PNG images) and structural elements like doors and windows.

To persist their work, I created a custom .2ds file format that stores room metadata. Users can save their floor plan and reload it later. The entire UI uses FlatLaf for a modern look instead of default Java styling."

6.1 Common Interview Questions & Strong Answers

6.1.1 Q1: "Why did you choose Java over Python or JavaScript?"

Technical Details

Strong Answer: "Java was the right choice for three reasons: First, I needed cross-platform desktop support—same code works on Windows, Mac, and Linux without rewriting. Second, I wanted modern cloud integration (Firebase, Google Auth), which are mature in Java. Third, Java's Swing framework is perfect for desktop UIs with grid layouts and real-time graphics.
Could I have used Python with Tkinter? Yes, but Swing is more powerful for complex UI. Could I use Electron (JavaScript)? Absolutely, but Java keeps deployment simpler—just one JAR file."

6.1.2 Q2: "How does the collision detection work?"

Technical Details

Strong Answer: "When placing a door on a wall, the code first checks: Does a window already exist at that position? I use simple interval overlap logic—if the distance between the door's center and window's center is less than the door length, there's overlap. I reject the placement and show an error.
Same logic applies to overlapping rooms: I check if the X/Y bounding boxes intersect. It's O(n) complexity but sufficient for typical floor plans."

6.1.3 Q3: "Why did you create a custom .2ds file format instead of JSON?"

Technical Details

Strong Answer: "Good question. Honestly, JSON would have been cleaner for maintainability. The .2ds format is custom and human-readable but verbose. At the time, I wanted something simple and specific to this domain—.2ds clearly means '2D Section.'
If I redo this, I'd switch to JSON for better tooling support and easier parsing. The trade-off: custom format is more domain-specific but harder to integrate with other tools."

6.1.4 Q4: "What authentication method do you recommend—Firebase or Google OAuth?"

Technical Details

Strong Answer: "For this app, both work, but they serve different purposes. Firebase stores user credentials locally—good for corporate/private deployments where you control the database. Google OAuth is better for public applications—users trust Google, and I don't store their passwords at all.
For a personal project, Firebase works. For a SaaS product, Google OAuth + optional Firebase backup would be ideal."

6.1.5 Q5: "How would you scale this to handle 1000 users saving plans simultaneously?"

Technical Details

Strong Answer: "Current architecture has a bottleneck: file I/O on the user's local disk. For scaling:

1. Move plan storage to cloud (Firebase Realtime Database or AWS S3)
2. Implement multi-user collaboration—use WebSockets to sync changes in real-time
3. Add database indexing for fast retrieval
4. Move the UI to web (React/Vue) instead of desktop to reduce deployment friction
5. Implement plan versioning and conflict resolution

The web version would be more scalable than the desktop app."

6.1.6 Q6: "What would you change if you rewrote this today?"

Technical Details

Strong Answer: "Several things:

1. Move to a web framework (React + Three.js) instead of Java Swing—better UX and easier deployment
2. Use WebGL for 3D capabilities—render 3D models of rooms alongside 2D floor plans
3. Implement real-time collaboration using Operational Transformation or CRDT
4. Use a proper database schema instead of custom file formats
5. Add undo/redo functionality (currently missing)
6. Add mobile support (Responsive web design) so users can edit plans on tablets

"

6.2 Key Technical Talking Points

Feature	Technical Detail
Grid-based positioning	Users don't think in pixels; they think in 1-meter, 2-meter units
Furniture images	Currently hardcoded paths—should be configurable for different asset libraries
Cloud saving	Plans are only saved locally—.2ds hints at "2D Section," but it's really just text
Collision detection	$O(n^2)$ for room overlap checks—acceptable for small plans
FlatLaf	3-line fix that modernizes the whole UI

Table 4: Key Technical Implementation Details

7 Simplification for Interviews

Information

If interviewer looks confused, simplify further:
"Think of it like PowerPoint but for floor plans:

- You open PowerPoint (Login screen)
- Click 'Insert' to add shapes → I click 'Add Room'
- Drag shapes around → I drag rooms on grid
- Save file → I click 'Download' to save as .2ds
- Open file later → Rooms reappear exactly where they were

The backend is just:

- A list of Room objects (position, size, color)
- A list of Furniture objects (position, type, image)
- A list of Doors/Windows (wall position, type)
- A file saver that writes these lists to a text file
- A file loader that reads the text and recreates the lists

That's 80% of what the app does. The other 20% is UI (buttons, colors) and authentication (login screen)."

8 Final Checklist

Information

You Should Be Able to Explain:

1. ✓What the app does (creates 2D floor plans visually)
2. ✓Why it's useful (quick prototyping, no expensive software needed)
3. ✓Why Java (desktop, cross-platform, cloud integration)
4. ✓What each main file does (Login, GoogleAuthConfig, RoomPlan2, Room class, Furniture class)
5. ✓How the save/load system works (custom .2ds format with room metadata)
6. ✓How collision detection prevents overlaps
7. ✓The user flow (Login → Create Rooms → Save → Reload)
8. ✓Why you'd chose different technologies today (Web app, 3D support, real-time collaboration)

9 Future Enhancements

9.1 Short-term Improvements

- **Undo/Redo Functionality:** Implement command pattern for action history
- **JSON File Format:** Replace .2ds with standard JSON for better interoperability
- **Export to PDF:** Allow users to export floor plans as printable documents
- **Measurement Tools:** Add rulers and dimension labels on the canvas

9.2 Long-term Vision

- **Web Version:** Migrate to React/Vue with WebGL for 3D visualization
- **Cloud Storage:** Store plans in Firebase/AWS for access from any device
- **Real-time Collaboration:** Multiple users editing same plan simultaneously
- **3D Preview:** Generate 3D walkthrough from 2D floor plan
- **Mobile App:** iOS and Android versions for on-site editing
- **AI Assistance:** Suggest optimal room arrangements based on constraints

10 Conclusion

Build2D demonstrates a complete desktop application lifecycle:

- **User Authentication:** Secure login with multiple methods
- **Core Functionality:** Room creation, manipulation, and decoration
- **Data Persistence:** Custom file format for saving and loading
- **Modern UI:** FlatLaf for professional appearance
- **Cross-platform:** Java ensures compatibility across operating systems

This project showcases practical software engineering skills including:

- Object-oriented design patterns
- Event-driven programming
- File I/O operations
- Third-party API integration (Firebase, Google OAuth)
- User interface design principles
- Collision detection algorithms

References

- Java Swing Documentation: https://docs.oracle.com/javase/tutorial/uiswing/
- FlatLaf Library: https://www.formdev.com/flatlaf/
- Firebase Admin SDK: https://firebase.google.com/docs/admin/setup
- Google OAuth 2.0: https://developers.google.com/identity/protocols/oauth2
- Apache Maven: https://maven.apache.org/

A Appendix: Complete File Structure

Code Example

```
Build2D/
          src/
                main/
                      java/
                            Login.java
                            GoogleAuthConfig.java
                            RoomPlan2.java
                            test.java
                      resources/
                          images/
                              bed.png
                              chair.png
                              sofa.png
                config/
                    client_secrets.json (not committed)
                    key.json (not committed)
          pom.xml
          README.md
          PRIVACY_POLICY.md
```

Listing 6: Project Directory Structure